C# Ders - 7 Ekim 2026 Ders Notları

1. Değişken Tanımlama Kuralları ve Standartlar

C# dilinde değişken tanımlama sözdizimi:
`VeriTipi DeğişkenAdı = Değer;`

Zorunlu Kurallar

- **Başlangıç Karakteri:** Değişken isimleri yalnızca bir **harf** veya alt çizgi (`_`) ile başlayabilir. **Asla sayı ile başlayamaz** (`1sayi` derleme hatası verir; `sayi1` veya `_sayi` geçerlidir).
- **Boşluk Yasağı:** Değişken adlarında boşluk kullanılamaz. Çok kelimeli isimlendirmelerde ya alt çizgi (`ogrenci_yasi`) ya da sektör standardı olan **camelCase** (`ogrencilerinYasOrtalamasi`) tercih edilir.
- **Büyük/Küçük Harf Duyarlılığı (Case-Sensitivity):** C# büyük ve küçük harfleri tamamen farklı algılar.
  - `int Number = 1;` ile `int numBer = 8;` bellekte iki ayrı değişkendir.
- **Aynı İsim Yasağı:** Veri tipi farklı olsa dahi **aynı kapsam (scope) içinde aynı isimle ikinci bir değişken tanımlanamaz**:C#

```
int age = 20;
// string age = "Yirmi"; // DERLEME HATASI! Tip farklı olsa da isim aynı olamaz.
```

2. Bellek Kavramları: Undefined, Empty ve Null Ayrımı

Bu üç kavram sıkça karıştırılır; aralarındaki yapısal farklar şunlardır:

Undefined (Tanımsız / İlklendirilmemiş)

- C#'ta yerel (local) bir değişken tanımlanıp değer atanmadığında (`int yas;`) durumudur.
- Derleyici bu değişkenin okunmasına izin vermez (*"Use of unassigned local variable"* hatası verir). Değeri henüz var olmamıştır.

Empty (Boş)

- Değişken bellekte tanımlıdır, somut bir alanı kaplar fakat **içeriği boştur**.
- En yaygın örneği metinlerdir:C#

```
string deger = string.Empty; // "" ile aynıdır.
```

- Belleğin heap bölgesinde 0 karakter uzunluğunda bir nesne tahsis edilmiştir. Güvenlidir; metot çağrıldığında program çökmez.

Null (Hiçlik / Adres Yok)

- Değişken tanımlıdır ancak bellekte **hiçbir nesneyi işaret etmez** (pointer boştur).

- **Önemli Kural:** Standart ilkel (primitive / value type) tipler (`int`, `double`, `bool` vb.) doğrudan `null` değer **alamaz**. `null` yalnızca referans tipler (`string`, nesneler) veya özel `Nullable<T>` (`int?`) tipler için geçerlidir.
- `null` olan bir değişkenin özelliği veya metodu çağrılırsa program `NullReferenceException` fırlatarak çöker.

3. İlkel Tipler ve Varsayılan Değerler (`default`)

C#'ta değişkenler değer almadığında veya `default` anahtar kelimesiyle üretildiğinde türe özgü taban değerlere döner (Java'daki yerel değişkenlerin aksine C# bu değerleri `default` anahtar sözcüğüyle açıkça sağlar).

Kategori	Veri Tipleri	Bellek Boyutu	Varsayılan Değeri (`default`)	Kritik Not
Tam Sayılar	`byte`, `short`, `int`, `long`	1B, 2B, 4B, 8B	`0`	**Risk:** Ekrana basılan `0` değeri, kullanıcının girdiği gerçek bir değer mi yoksa sistemin atadığı varsayılan değer mi ayırt etmek zordur.
Ondalıklılar	`float`, `double`, `decimal`	4B, 8B, 16B	`0.0`	`float` için `f`, `decimal` için `m` eki zorunludur.
Mantıksal	`bool`	1 Byte	`false`	Yalnızca `true` / `false`.
Karakter	`char`	2 Byte	`'\0'` (Null char)	Karakterin ASCII 0 karşılığıdır, ekranda boş görünür.
Referans	`string` ve nesneler	Değişken	`null`	Bellekte herhangi bir adresi göstermez.

4. `var` ve `const` Kullanımı

`var` (Örtük Tür Çıkarımı)

- Dinamik (`dynamic`) bir tip **değildir**; C#'ın tip güvenliğini (type safety) bozmaz.
- Değişkenin türü, eşitliğin sağ tarafındaki değerden **derleme anında (compile-time)** belirlenir.C#

```
var isim = "Ahmet"; // Derleyici bunu kesin olarak 'string' yapar.
var sayi = 45;      // Derleyici bunu kesin olarak 'int' yapar.
```

- Belirlenen tür sonradan değiştirilemez (örn: `sayi = "Metin";` hata verir).

`const` (Sabitler)

- Program boyunca değeri **asla değiştirilemeyecek** değerler için tanımlanır.
- Tanımlandığı anda ilk değeri verilmek zorundadır.
- Kodlama standardı olarak değişken ismi **tamamı BÜYÜK HARFLERLE ve alt çizgi (`_`)** ile yazılır:C#

```
const double PI_SAYISI = 3.14159;
// PI_SAYISI = 3.14; // DERLEME HATASI! Değer değiştirilemez.
```

5. Tür Dönüşümleri (Type Casting & Conversion)

C#'ta veriler üç farklı yöntemle dönüştürülür:

A. Örtük (Implicit) Dönüşüm

Küçük veri türünden büyük veri türüne geçerken veri kaybı riski olmadığı için derleyici otomatik yapar:

C#

```
int kucukSayi = 513;
long buyukSayi = kucukSayi; // int (4 byte) -> long (8 byte) problemsiz genişler.
```

B. Açık (Explicit / Cast) Dönüşüm

Veri kaybı veya taşma (overflow) riski olduğunda parantez içinde `(HedefTip)` yazılarak zorla yapılır:

C#

```
double pi = 3.94;
int piTamsayi = (int)pi; // Sonuç: 3 (ondalık kısım yuvarlanmaz, doğrudan kesilip atılır).

int sayi = 513;
byte baytSayi = (byte)sayi; // Sonuç: 1
```

> **Bellek Taşması Notu:** `byte` türü en fazla `255` tutabilir. `513` değeri `256` modülüne girer (`513 % 256 = 1`). Yüksek basamaklardaki bitler çöpe atıldığı için veri bozulur ve ekrana `1` basılır.

C. Parse, TryParse ve Convert Farkları

Yöntem	Sözdizimi	Başarısız Olursa (Harf girilirse)	Kullanım Amacı
`Parse`	`int.Parse("123")`	**Uygulama çöker**(`FormatException`)	Girişin kesin olarak sayı olduğundan emin olunduğunda.
`Convert`	`Convert.ToInt32("123")`	Değer `null` ise `0` döner; format hatalıysa **çöker**	`double`'ı `int`'e çevirirken yuvarlama yapar (`3.94` → `4`).
`TryParse`	`int.TryParse(girdi, out int sonuc)`	**Çökmez!** `false` döner, değişkene `0` atar	Konsol/kullanıcı girişlerinde standart güvenlik yöntemi.

6. Aritmetik İşlemler ve Öncelik Kuralları

Ön Ek / Son Ek Operatörleri (`++x` vs `x++`)

- **Prefix (`++x` / `-x`):** Değişkenin değeri o işlem satırında **önce artırılır/azaltılır**, ardından formüle dahil edilir.

- **Postfix (`x++` / `x--`):** Değişkenin **mevcut değeri** o anki işlemde kullanılır, işlem tamamlandıktan sonra değeri 1 artırılır/azaltılır.

C#

```
int n = 5;
int y = 3;

Console.WriteLine(++n * --y);
// 1. n hemen 6 olur.
// 2. y hemen 2 olur.
// 3. 6 * 2 = 12 ekrana yazdırılır.

Console.WriteLine(n++);
// Ekrana mevcut değer olan 6 basılır.
// Satır bittikten hemen sonra n arka planda 7 olur.

Console.WriteLine(++n);
// n hemen 1 artırılır (8) ve ekrana 8 basılır.
```

Bölme İşleminde Hassasiyet Kaybı

C#

```
int n = 5, y = 3;
double sonuc = (double)y / n; // 3.0 / 5 = 0.6
```

İki tam sayı (`int / int`) bölündüğünde sonuç yine bir tam sayı olur (`3 / 5 = 0`). Virgülden sonraki değerin korunabilmesi için değişkenlerden en az birinin başına `(double)` veya `(float)` getirilerek cast edilmesi şarttır.

Mod Alma Operatörü (`%`)

C#

```
int kalan = 5 % 3; // Sonuç: 2 (5'in 3'e bölümünden kalan değer)
```

7. Kaçış Karakterleri ve Format Belirleyiciler

- `\n`: Yeni satıra geçer.
- `\t`: Metinler arasına 1 sekme (tab) boşluk bırakır, verileri hizalar.
- `\r`: İmleci satır başına taşır (üzerine yazma işlemleri için).
- `\"`: Metin bloğu içerisinde çift tırnak karakteri basar.
- `:c2`: Sayıyı yerel para formatına dönüştürür (örn: `₺350,00`).
- `:p`: Yüzde oranına çevirir (örn: `0.15` → `%15`).
- `:D8`: Tam sayının başına toplam uzunluk 8 olana kadar `0` ekler (örn: `00000123`).